

Universidad Nacional Autónoma de México

FACULTAD DE CIENCIAS

CHAT MULTICLIENTE

Primer proyecto de Modelado y Programación

Autor:

Evan Miranda Martínez

Septiembre 2026

1. Resumen

Este trabajo presenta **Gabby**, un sistema de chat multicliente con arquitectura cliente-servidor, desarrollado como primer proyecto de la materia **Modelado y Programación** de la licenciatura en **Ciencias de la Computación** de la **Facultad de Ciencias, UNAM**. El documento aborda tanto el modelado del sistema como su implementación: un cliente escrito en **Go**, con una interfaz de terminal construida con la popular biblioteca **Bubble Tea** bajo la **Arquitectura Elm**, y un servidor escrito en **Rust** y **Tokio**, sometido a pruebas de carga con 3,000 usuarios simultáneos.

2. Introducción

Objetivo

El objetivo general de este proyecto es diseñar e implementar un sistema de **Chat Multicliente** basado en la arquitectura cliente-servidor, que permita la comunicación en tiempo real entre múltiples clientes.

Alcance

El alcance de este proyecto está delimitado en gran parte por el protocolo establecido en clase. En términos funcionales, se trata de un chat convencional con soporte para **mensajes públicos**, **salas** y **mensajes privados**. No obstante, el alcance de **Gabby** se amplió más allá de lo requerido, incorporando además una interfaz de usuario moderna por terminal, pensando en mejorar la experiencia de usuario.

Sin embargo, **Gabby** sigue siendo bastante limitado en comparación a muchísimas de las alternativas disponibles. Especialmente debido a que no contiene, **cifrado**, **persistencia de datos**, **sistema de autenticación**, o **envío de archivos multimedia**.

3. Análisis y Diseño

El **Chat Multicliente** puede dividirse fundamentalmente en dos programas independientes: **cliente** y **servidor**. La comunicación entre ambas partes se realiza con **JSON** sobre **TCP** por medio del protocolo establecido en el curso. El protocolo cuenta con **23** mensajes establecidos, más un mensaje genérico de respuesta por parte del servidor.

Cliente

El cliente se diseñó para ser desarrollado en el lenguaje de programación **Golang**, utilizando la biblioteca **Bubble Tea** para la interfaz de terminal. Esta decisión se tomó de manera previa al análisis de la especificación del proyecto, pues estaba realmente entusiasmado en utilizar la biblioteca para algún proyecto. En consecuencia, **Gabby** adopta la **Arquitectura ELM** (*Model-Update-View*) como arquitectura principal.

La **Arquitectura ELM** surge en la comunidad del lenguaje de programación funcional **ELM** como una manera de estructurar interfaces gráficas reactivas y funcionales. Se compone de estos tres elementos:

Modelo	Representa el estado actual de la aplicación.
Update	Ciclo que actualiza el Modelo a partir de mensajes recibidos.
Vista	Una fotografía del estado en un momento dado.

La arquitectura mantiene un flujo de datos unidireccional: **Modelo** $\Rightarrow$ **Update** $\Rightarrow$ **Vista** $\Rightarrow$ **Modelo**. Y permite la comunicación entre componentes y etapas gracias a dos abstracciones: los **mensajes**, tipos de datos que notifican la ocurrencia de eventos, y los **comandos**, indicaciones para ejecutar acciones con efectos secundarios.

Generalmente, las aplicaciones realizadas a partir de la **Arquitectura ELM** cuentan con un modelo raíz que encapsula varios submodelos. Cada submodelo se encarga de la lógica de una parte específica y delimitada de la interfaz manteniendo su propio estado, su propia función de actualización y su propia vista. El modelo raíz orquesta estos submodelos: delega los mensajes correspondientes a cada uno, integra sus vistas parciales en la vista general de la aplicación, y coordina la comunicación entre ellos cuando es necesario.

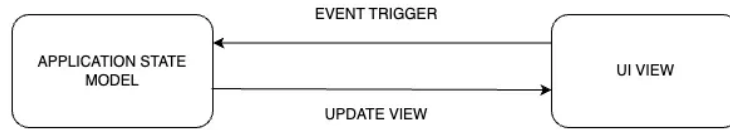


Figura 1: Flujo de datos de la arquitectura ELM

Interfaz

La interfaz de **Gabby** está fuertemente inspirada en interfaces de terminal populares, como **Lazygit** y **Lazydocker**. Por lo tanto, tanto la experiencia de usuario como el diseño visual retoman los recursos característicos de esta filosofía, es decir: distribución de la información en paneles, formularios modales para la captura de datos, y una navegación por atajos de teclado modal y basada en aspectos mnemotécnicos.

Esto es bastante visible en la figura 2. En ella podemos observar varias de las características antes mencionadas.

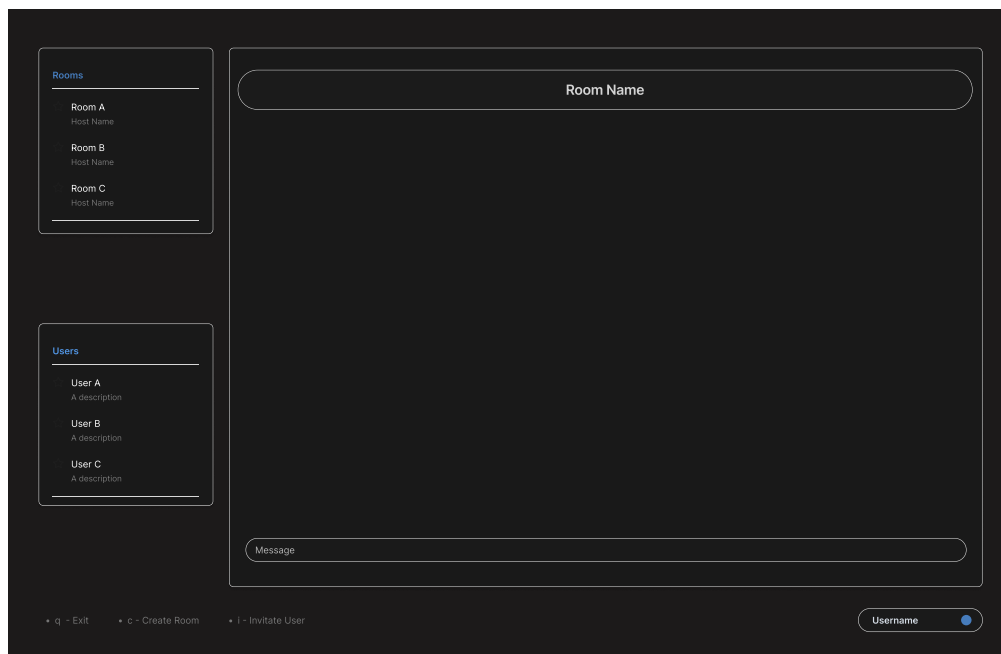


Figura 2: Mockup de la interfaz de terminal del cliente.

La vista principal se compone de cuatro paneles persistentes organizados en pantalla:

Panel	Descripción
Panel de cuartos	Permite visualizar las salas a las que pertenece el usuario.
Panel de usuarios	Despliega la lista de usuarios conectados en el servidor.
Panel de chat	Es el panel principal de la aplicación. Contiene un <i>viewport</i> para el scroll y visualización de texto, y un <i>textinput</i> para manejar el envío de mensajes.
Footer contextual	Ubicado en la parte inferior de la pantalla, actualiza dinámicamente los atajos de teclado disponibles y muestra el estado actual del cliente.

Cuadro 1: Componentes de la vista principal del cliente.

Además, para realizar la captura de datos y decisiones del cliente se opta por utilizar vistas **modales**. Es decir, interfaces superpuestas al contenido principal que bloquean temporalmente la interacción con el resto de la aplicación hasta que el usuario realiza una acción o la cierra.

El diseño de la interfaz de **Gabby** utiliza paneles genéricos embebidos en el **Modelo** principal. De esta

Modal	Descripción
Input Modal	Captura cadenas de texto del usuario.
List Modal	Permite la selección simple o múltiple de los items mostrados por una lista.
Confirm Modal	Permite ejecutar operaciones binarias excluyentes.
Notification Modal	Despliega mensajes de notificación.
Wizard Modal	Presenta flujos secuenciales de múltiples pasos con estado compartido.

Cuadro 2: Modales utilizados en el cliente.

manera se desacoplan los tipos de datos de **dominio** de la lógica visual, permitiendo, por ejemplo, reutilizar componentes entre distintos paneles. Esto se puede observar en el diagrama de la figura 3 con el modelo de lista y los paneles tanto de usuarios como de salas.

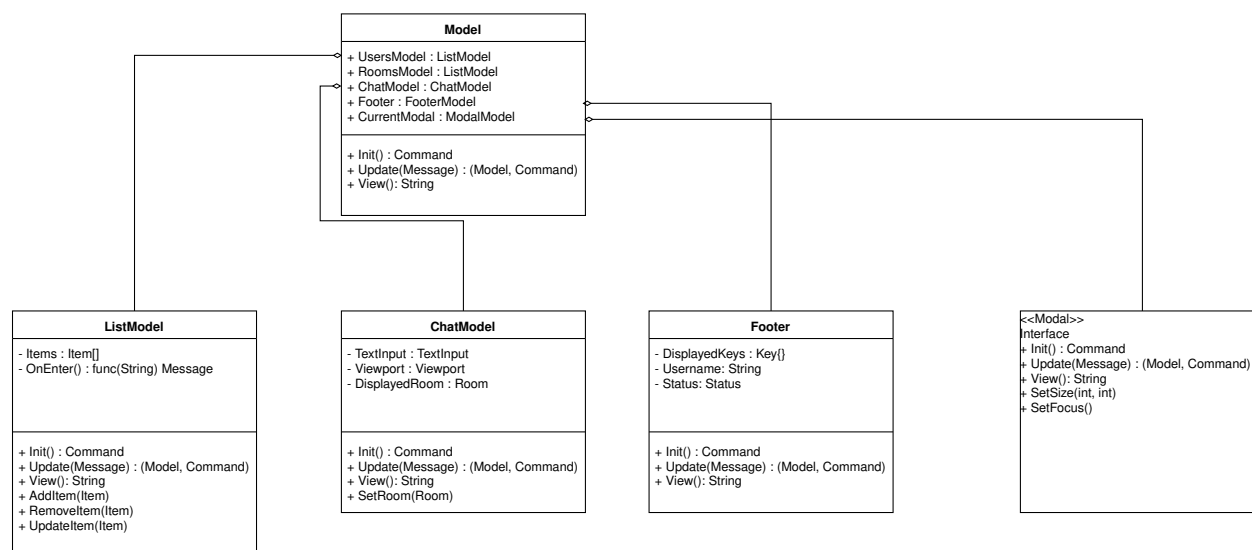


Figura 3: Diagrama UML de la organización de paneles.

Además, el diseño se simplifica gracias a los componentes prefabricados de **Bubbles**, la biblioteca hermana de **Bubble Tea**. Específicamente, **Bubbles** ofrece tres componentes esenciales para **Gabby**: **viewport** (permite el scroll del historial de mensajes), **textinput** (maneja la captura de texto) y **footer**, convirtiendo a algunos de los modelos principales en, básicamente, *wrappers* sobre estos componentes.

La interacción de los modales con la vista principal depende de la implementación de la interfaz **Modal**. El polimorfismo de modales permite abstraer su comportamiento común y simplificar la visualización y el control de cada tipo de panel modal. Principalmente, nos permite modelar la interfaz como una máquina de estados, donde la existencia de un **CurrentModal** activo determina el estado de **captura**: mientras un modal esté presente, el modelo raíz le delega la totalidad de los mensajes de entrada, secuestrando el control de la interfaz hasta que el modal se resuelve y **CurrentModal** vuelve a quedar vacío.

Modelos de Dominio

La presentación de la interfaz depende en gran medida de los datos mostrados. Es bastante intuitivo modelar los datos básicos dado el contexto: **User** y **Room**. El usuario guarda un username y un estado. La sala guarda un roomname, una lista de usuarios y un historial de mensajes (dado que el servidor no tiene persistencia). Precisamente por eso, **Session** almacena en memoria el estado de un cliente, es decir su identidad, sus salas disponibles, los usuarios conectados, las salas globales y los mensajes privados.

Capa de Red

Cualquier aplicación de terminal realizada con **Bubble Tea** opera en un único hilo de renderizado. Por lo tanto, intentar leer el socket TCP mediante el mismo hilo de ejecución provocaría que la aplicación se

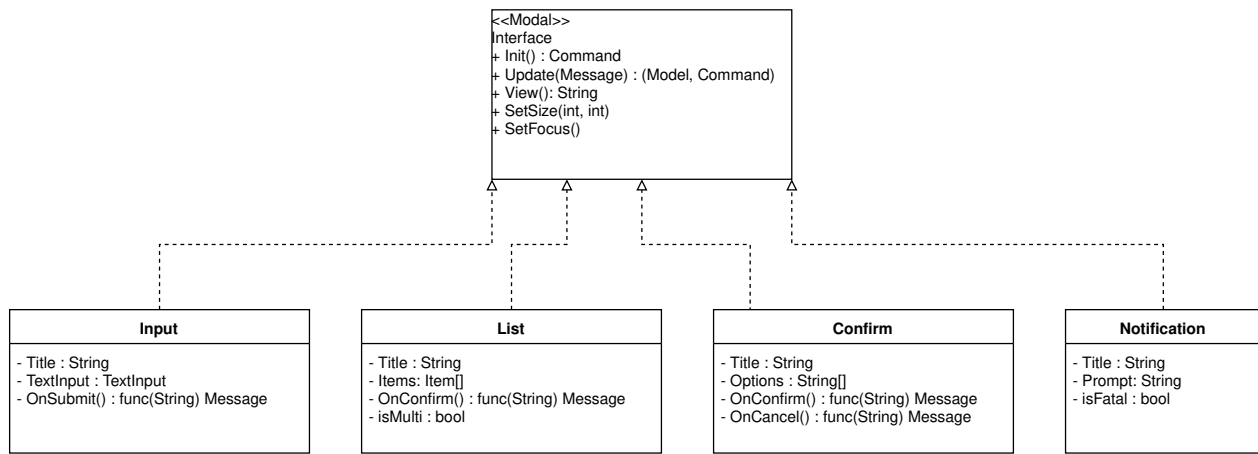


Figura 4: Diagrama UML de la organización de modales.

bloqueara por completo.

Go tiene un principio bastante interesante acerca de la comunicación asíncrona: *“Do not communicate by sharing memory; instead, share memory by communicating”*. Esto se logra a través de canales que permiten para pasar referencias a datos entre goroutines.

De esta manera podemos utilizar una goroutine independiente para leer los datos del socket, comunicar las lecturas a un canal y utilizar los **cmd** de **Bubble Tea** para inyectar los **messages** en la función **Update** del Modelo raíz. Este flujo se puede observar de mejor manera en el diagrama 5.

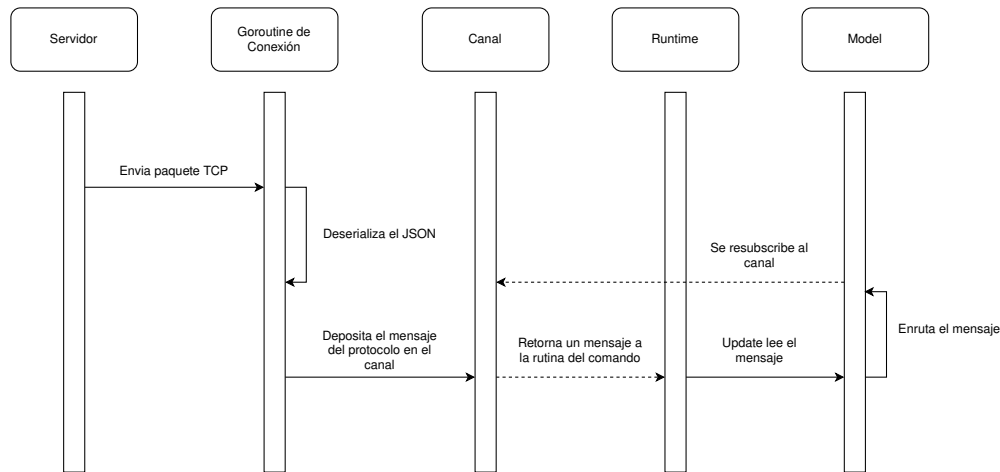


Figura 5: Diagrama de secuencia de la interacción por red.

Enrutamiento

El método **Update** del modelo raíz centraliza el despacho de eventos clasificándolos en tres dominios principales.

Mensajes del Servidor

Representa los mensajes y respuestas entrantes del servidor.

Mensajes del Cliente

Representa los mensajes enviados por el cliente.

Mensajes Internos

Representa mensajes de cambio interno en la interfaz.

Cada uno de ellos cuenta con un **Router** que mapea cada tipo de mensaje a una función *handler*.

Servidor

Fearless Concurrency es el principio fundamental de la elección del lenguaje de programación **Rust** para el desarrollo del servidor. Muchas de las implementaciones de seguridad de memoria que incluye el lenguaje, mediante su sistema de *ownership* y *borrowing* permiten transformar errores en tiempo de ejecución a errores en tiempo de compilación. De esta manera, *data races* y otros errores comunes en programación concurrente, que en otros lenguajes solo se manifiestan durante la ejecución, son detectados por el compilador antes de que el programa siquiera se ejecute. Esto permite escribir código concurrente con la confianza de que, si compila, está libre de una clase entera de errores que tradicionalmente son difíciles de depurar y reproducir.

El modelo de programación asíncrona de **Rust** presenta una particularidad importante: tanto la sintaxis **async/await** como el **trait Future** están definidos a nivel del lenguaje y su biblioteca estándar, pero se trata de implementaciones **lazy** que no se ejecutan por sí solas, ya que **Rust** no incluye un entorno de ejecución asíncrono. Para esto, utilizamos **Tokio**, el entorno de ejecución asíncrono más popular dentro de la comunidad de **Rust**.

El entorno de ejecución de **Tokio** es bastante rápido debido a su arquitectura basada en un diseño modular y de alto rendimiento enfocado en el I/O no bloqueante. Se basa principalmente en el patrón de diseño **Reactor** que permite manejar múltiples **tasks** en un solo hilo de ejecución, con la ayuda de un **task scheduler**, un **I/O Driver** y **timers** que se comunican directamente con el sistema operativo.

Tokio también ofrece varias abstracciones de alto nivel y macros que permiten trabajar fácilmente la programación asíncrona.

Arquitectura

El estado de la aplicación se centraliza en una única estructura de estado global (**Hub**) que administra a todos los clientes, salas públicas y salas privadas activas en el servidor. Internamente, el **Hub** almacena estas entidades en diccionarios (**HashMap**), indexados por el nombre de usuario o de sala, lo que permite consultas en tiempo $O(1)$ amortizado.

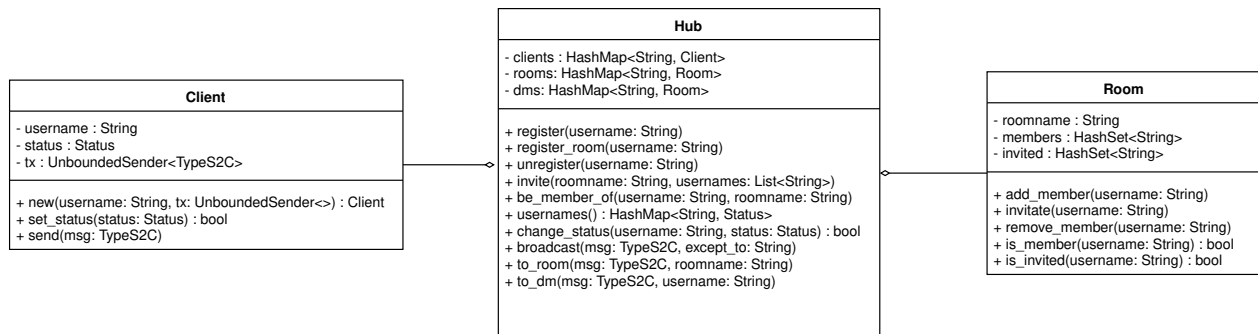


Figura 6: Diagrama UML del servidor.

Además, como puede observarse en el diagrama de la figura 6, el **Hub** funciona como mediador entre los múltiples clientes conectados y es el encargado de ejecutar la mayoría de las operaciones definidas en el protocolo. De esta manera, no existe comunicación directa entre clientes (**P2P**), ya que toda interacción debe pasar necesariamente a través del **Hub**.

Las salas mantienen su propio estado internamente, utilizando conjuntos (**HashSet**) para almacenar tanto a sus miembros como a sus invitados. De esta manera, se simplifica la lógica encargada de enrutar mensajes y eventos hacia las salas correspondientes.

Conexiones

Cada vez que entra un nuevo cliente al servidor, **Tokio** delega la conexión del socket a un **task** independiente, que se encarga de manejar la comunicación con el usuario. El primer paso de la comunicación siempre consiste en identificarse; si la tarea verifica la correctitud de la identificación, crea un **Client** y lo registra en el **Hub**.

En la figura 6, podemos notar que los **Client** no guardan una referencia del socket, si no que guardan un canal MPSC (*Multiple Producer, Single Consumer*). El canal permite desacoplar las tareas de I/O

relacionadas al socket, y permitir que cada conexión se gestione sola.

Después de la identificación, la **task** entra en un bucle que multiplexa dos fuentes de eventos asíncronos.

1. **Lectura del Socket:** Escucha los mensajes que llegan a través del socket TCP, deserializa el mensaje y lo transfiere al **enrutador** para ejecutar la acción correspondiente.
2. **Lectura del Canal:** Escucha los mensajes entrantes al canal, extra el mensaje y lo transmite por el socket TCP hacia el cliente.

Mediante este mecanismo, el runtime suspende la tarea hasta que alguno de los dos *futures* esté listo, procesando de inmediato el primer evento disponible.

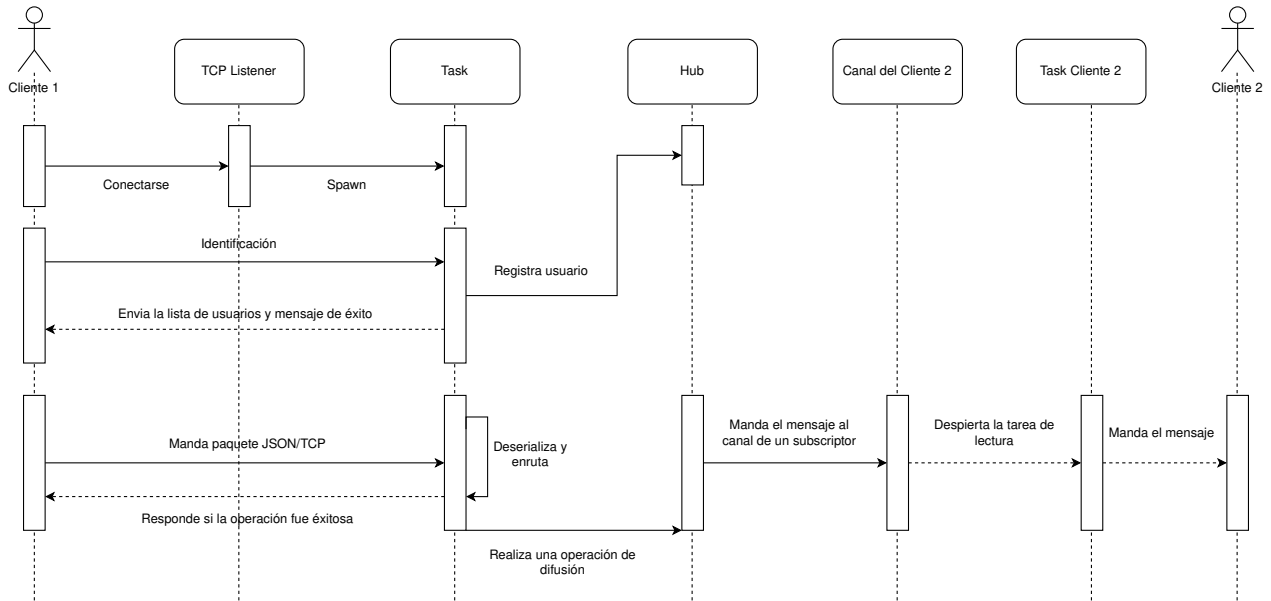


Figura 7: Diagrama de secuencia del servidor.

Cada conexión necesita acceder y mutar concurrentemente el estado global del sistema. El **Hub** se comparte mediante múltiples tareas por medio de **exclusión mutua**, que permiten modificar el estado compartido de manera segura.

Enrutamiento y Protocolo

Rust permite modelar información mediante **tipos de datos suma** a través de sus enumeraciones. Un tipo suma representa un valor que solo puede adoptar una de varias formas posibles, donde cada variante puede llevar asociados datos de tipos distintos entre sí. Los **tipos de datos suma**, ofrecen importantes ventajas de seguridad de tipos, en particular la posibilidad de realizar **pattern matching** exhaustivo, es decir un manejo explícito de todos los casos con verificación del compilador.

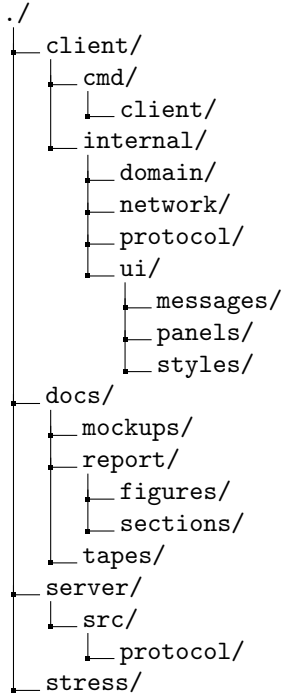
El **protocolo** del servidor modela la comunicación mediante dos tipos suma: **ClientMessage**, que representa los mensajes enviados del cliente al servidor, y **ServerMessage**, que representa los mensajes enviados del servidor al cliente. Ambos tipos de dato, se serializan y deserializan con la biblioteca de **serde**, por medio del etiquetado interno.

Los mensajes **deserializados** válidos se delegan a una función **router**, la cual realiza internamente un *match* sobre el **ClientMessage**. El **pattern matching** permite mapear fácilmente cada variante a una función auxiliar **handler**, encargada de manejar la operación correspondiente, típicamente modificando el estado global y difundiendo mensajes **ServerMessage** al resto de los clientes.

4. Implementación

Estructura

Dado el alcance del proyecto, la organización del repositorio se realizó mediante un **monorepo**. Lo anterior, permite centralizar el código tanto del cliente, como del servidor, y orquestar tareas comunes de desarrollo.



Por un lado, el **Client** se organiza siguiendo la Standard Go Project Layout, la organización estándar de proyectos en **Go**. Utiliza **cmd** para el punto de entrada del cliente e **internal** para organizar el código interno, relacionado tanto con la lógica de negocio como con la vista de usuario.

Por otro lado, el **servidor** se organiza de una manera más simple, en un único **crate** que centraliza el comportamiento general de la aplicación. Únicamente el protocolo se separa en su propio módulo.

Dependencias

Las dependencias del desarrollo se listan en la la tabla 3.

Componente	Versión
Go	1.26.6
Bubble Tea	1.3.10
Lipgloss	1.1.0
Bubbles	1.0.0
Rust	1.97.1
Tokio	1.53.1
Serde	1.0.229
Serde JSON	1.0.151
Just	1.57.0
Prek	0.5.0
k6	2.3.0
xk6-tcp	main

Cuadro 3: Requisitos y dependencias del proyecto.

Flujo de Trabajo

Para unificar los sistemas de compilación, ejecución de pruebas unitarias, documentación y ejecución de ambos programas, se utilizó **Just**. Por ejemplo, el siguiente comando realiza el build tanto del cliente como del servidor.

```
1 just build
```

Además, para mantener una consistencia de código se utiliza **Prek** para realizar un formateo del código, tanto de **Rust** como de **Go**, antes de cada commit.

La mayoría del flujo inicial se basó en el flujo de trabajo *code a little, test a little*. Las pruebas unitarias se ejecutan en **workflows** de **integración continua** durante cada push.

Servidor

Observemos como se translada el modelo del servidor al código.

Multiplexado

La macro `tokio::select!` permite ejecutar concurrentemente varias expresiones en la misma **task**. De esta manera, ambas fuentes de eventos alternan su ejecución dependiendo del cumplimiento de las ramas.

```
1 tokio::select! {  
2     // 1. Lectura del socket TCP  
3     result = reader.read_line(&mut line) => {  
4         match result {  
5             Ok(0) => break, // EOF  
6             Ok(_) => {  
7                 if let Ok(msg) = serializer::deserialize(&line) {  
8                     if let Err(err) = route_msg(msg, &_username, &hub, &mut writer).await {  
9                         send_msg(&mut writer, new_response(TypeC2S::Invalid, err, None)).  
10  
11                     }  
12                 }  
13             }  
14             Err(_) => break,  
15         }  
16     }  
17  
18     // 2. Recepcion de mensajes salientes desde el canal MPSC  
19     Some(msg) = rx.recv() => {  
20         send_msg(&mut writer, msg).await;  
21     }  
22  
23     else => break,  
24 }
```

Enrutamiento

La función de enrutamiento implementa el patrón descrito en la Sección 3: recibe un **ClientMessage** ya deserializado y, mediante un **match** exhaustivo sobre sus variantes, delega la operación correspondiente a una función *handler* específica.

```
1 async fn route_msg<W>(  
2     msg: ClientMessage,  
3     username: &str,  
4     hub: &Arc<Mutex<Hub>>,  
5     writer: &mut W,  
6 ) -> Result<(), MessageResult>  
7 where  
8     W: AsyncWrite + Unpin,  
9 {  
10     match msg {  
11         ClientMessage::Users => handle_users_list(hub, writer).await,  
12         ClientMessage::Status { status } => handle_status(username, status, hub).await,  
13         ClientMessage::PublicText { text } => handle_public_text(username, &text, hub).await  
14     },  
15 }
```

```

14     ClientMessage::NewRoom { roomname } => {
15         handle_create_room(username, &roomname, hub, writer).await?;
16     }
17     ...
18 }
19 }

```

Estado Compartido

La mayoría de funciones `handler` recibe un `&Arc<Mutex<Hub>>`. Está es una manera popular en la comunidad de **Rust** para implementar el **estado compartido por exclusión mutua** para realizar modificaciones en un estado global. Sin embargo, la cautela al abrir el bloqueo tiene que realizarse minuciosamente y abarcando la menor cantidad de operaciones.

Por ejemplo, apoyarse ampliamente de *scopes* resulta indispensable.

```

1 let result = {
2     let hub = hub.lock().unwrap();
3     hub.send_to(&msg, sender, receiver)
4     // El hub se libera al salir de este scope
5 };
6
7 match result {
8     Ok(_) => {}
9     Err(e) => {
10         send_msg(
11             writer,
12             new_response(TypeC2S::Text, e, Some(receiver.to_string())),
13         )
14         .await;
15     }
16 }

```

Cliente

Concurrencia en el Cliente

Mantener sincronizada la interfaz de usuario con el estado global del servidor sin bloquear el hilo de renderizado es una tarea indispensable para aplicaciones con interfaces visuales. Por esta razón, la concurrencia resulta igual de necesaria en la capa del **cliente** que en **servidor**; no quieres ofrecer una experiencia de usuario lenta y poco responsiva.

La conexión se encuentra desacoplada de la interfaz y corre en una **goroutine** secundaria. Por cada línea que lee del buffer, si no está malformada, se deserializa y se envía a un canal.

```

1 // Listen escucha los mensajes del servidor al cliente y los manda a traves de un canal
2 func (manager *ConnectionManager) Listen() {
3     for {
4         line, err := manager.reader.ReadString('\n')
5         if err != nil {
6             log.Printf("Connection closed: %v", err)
7             return
8         }
9
10        msg, err := protocol.Deserialize(line)
11
12        if err != nil {
13            log.Printf("Invalid message: %s", line)
14            continue
15        }
16
17        manager.incoming <- msg
18    }
19 }

```

El modelo principal espera mensajes de ese canal. Cuando recibe alguno, retorna un comando con el mensaje ya deserializado.

```

1 func (m Model) waitForServerMsg() tea.Cmd {

```

```

2     return func() tea.Msg {
3         return <-m.conn.Messages()
4     }
5 }

```

El mensaje es capturado por **Update** y delegado al *router* de mensajes del servidor. Una vez procesado, el modelo se resubscribe al canal en espera de más mensajes.

```

1 func (m Model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
2     switch msg := msg.(type) {
3
4     case protocol.ClientMessage:
5         return m, m.routeClientMessage(msg)
6
7     case protocol.ServerMessage:
8         cmd := m.routeServerMessage(msg)
9         // Se resubscribe al canal
10        return m, tea.Batch(cmd, m.waitForServerMsg())
11
12    }
13    return m, nil
14 }

```

Modales

Los modales utilizados en la interfaz por terminal implementan la interfaz descrita en la Figura 4. Cada tipo de modal (Input, List, Confirm, Notification) satisface esta interfaz, lo que permite al modelo raíz tratarlos de manera uniforme.

El modelo raíz contiene un campo `activeModal` que implementa la máquina de estados descrita en el diseño: mientras este campo esté activo, se le delega la totalidad del flujo de entrada hasta resolverse.

```

1 type Modal interface {
2     InputCapturer
3     Init() tea.Cmd
4     Update(msg tea.Msg) (Modal, tea.Cmd)
5     View() string
6     Focus() tea.Cmd
7     SetSize(width, height int)
8 }

```

Por ejemplo, un evento puede invocar la creación de un modal y establecerlo en la vista principal.

```

1 case "c":
2     return m.openModal(panels.NewInputModal("Create Room", "Roomname", 16, func(value string) tea.Msg {
3         room, _ := protocol.NewRoomMessage(value)
4         return room
5     }, true))

```

La función `openModal` realiza la transición de estado *sin modal* a *modal activo*, y espera a que el modal se resuelva en algún momento posterior.

```

1 func (m *Model) openModal(modal panels.Modal) tea.Cmd {
2     modal.SetSize(m.width, max(0, m.height-3))
3     m.chatModel.Blur()
4     cmd := modal.Focus()
5     m.activeModal = modal
6     // Inicializa el modal
7     return tea.Batch(cmd, modal.Init())
8 }

```

Cuando el servidor responde exitosamente a la operación solicitada por el modal, el enrutador de mensajes del servidor resuelve el flujo: cierra el modal activo y actualiza el estado local con la nueva información.

```

1 if msg.Result == protocol.SUCCESS {
2     m.closeModal()
3
4     room := m.session.AddRoom(domain.RoomChannel, msg.Extra)
5     m.roomsModel.AddItem(NewRoomItem(room))
6 }

```

Finalmente, `closeModal` realiza la transición inversa, devolviendo la interfaz al estado sin captura.

```

1 func (m *Model) closeModal() {
2     m.activeModal = nil
3     m.syncFocus()
4 }

```

Resultados

Agregado a las pruebas unitarias, se crearon pruebas de estrés mediante una extensión de **K6** personalizada para TCP, **xk6-tcp**. Las pruebas simulan un flujo concurrente de alta carga:

1. Conecta **3000** usuarios distribuidos en varias etapas.
2. Cada usuario envía 5 mensajes públicos y espera 10 segundos.
3. El usuario se desconecta.

Los resultados obtenidos, se muestran en la siguiente tabla 4.

Métrica	Total	Promedio
Pico de usuarios concurrentes	3,000	–
Conexiones TCP totales	18,015	146.58 conn/s
Errores de conexión	0	–
Escrituras TCP	124,933	1,016.54 op/s
Lecturas TCP	3,093,867	25,173.77 op/s
Datos enviados por clientes	7.2 MB	59 kB/s
Datos distribuidos por servidor	524 MB	4.3 MB/s

Cuadro 4: Resultados de la prueba de estrés TCP.

5. Conclusiones

Se logró implementar un sistema de chat multicliente basado en la arquitectura cliente-servidor, cubriendo por completo el protocolo establecido en clase, además de incorporar una interfaz de usuario por terminal.

El servidor se desempeñó de manera bastante decente incluso bajo una carga considerable. Aunque esperaba que el sistema colapsara con un número mucho menor de usuarios concurrentes, los resultados muestran que soportó 3,000 usuarios simultáneos con una tasa de error prácticamente nula, al menos en el escenario simulado por las pruebas de estrés. No obstante, las pruebas tuvieron sus propias limitaciones al ejecutarse en un entorno local y no me permitieron detectar errores reales, como por ejemplo problemas en mi **framing**.

El proceso de desarrollo estuvo lleno de altibajos y de una carga de nuevos conocimientos abismal. En retrospectiva, creo haber tomado algunas decisiones técnicas sobreestimando mi capacidad de aprendizaje y el tiempo disponible. Por ejemplo, desarrollar el proyecto en dos lenguajes con los que tenía muy poca experiencia previa, o dedicar una cantidad considerable de tiempo a módulos no estrictamente necesarios, como el diseño detallado de la interfaz de terminal.

Precisamente, el tener que aprender tantos conceptos y tecnologías nuevas a la vez limitó el tiempo disponible para el modelado inicial, lo cual me llevó a tomar decisiones subóptimas para resolver ciertos problemas, mismas que terminé pagando con refactorizaciones.

Ahora que comprendo mejor las tecnologías con las que trabajé, espero poder modelar y anticipar mejor los próximos proyectos. En particular, pretendo delimitar el alcance desde el inicio y administrar mejor mis módulos de trabajo, para no invertir tanto tiempo arreglando problemas diagnósticables a partir de un buen diseño.

Referencias

- [1] Thomas Bandt. *Model-View-Update (MVU) – How Does It Work?* <https://thomasbandt.com/model-view-update>. 2020.
- [2] bashbunni. *Build a Terminal Wizard in Go (Bubble Tea Tutorial)*. <https://www.youtube.com/watch?v=G131diSVP8M&t=1229s>. 2023.

- [3] Charm. *Bubbles*. <https://github.com/charmbracelet/bubbles>. 2026.
- [4] Andrew Gerand. *Share Memory By Communicating*. 2010. URL: <https://go.dev/blog/codelab-share>.
- [5] Uzair Jawaid. *Understanding The Elm Architecture*. https://medium.com/@uzair-jawaid_26268/understanding-the-elm-architecture-f64eb8922ae9. 2025.
- [6] Piotr Sarnacki. *Arc and Mutex in Rust*. <https://itsallaboutthebit.com/arc-mutex/>. 2022.
- [7] Carol Nichols Steve Klabnik y Chris Krycho. *The Rust Programming Language*. 2026. URL: <https://doc.rust-lang.org/stable/book/index.html>.
- [8] Tokio. *Tutorial*. <https://tokio.rs/tokio/tutorial>.
- [9] Flo Woelki. *Building a TCP Echo Server in Rust is easy*. <https://www.youtube.com/watch?v=EFxSVqbuAUE&t=427s>. 2025.